Reviewer Pack — Algebraic Topology of Knot Invariants vs Resolution Proof Le…

Entry 175abe514ed4 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-25 08:46:32 UTC

Algebraic Topology of Knot Invariants vs Resolution Proof Length for Tseitin Formulas

Entry ID: `175abe514ed4` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-25 08:46:32 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Topology (Knot Theory) **Field B** (complexity object): Complexity Theory: Resolution Proof Complexity (for Tseitin Formulas)

Statement:

> For every n-vertex graph G, the number of Seifert matrices corresponding to its knots is upper bounded by 2^n and there exists a polynomial-time computable function f(n) such that the resolution proof length for the Tseitin formula on G is at least f(n).

Rationale (proposer's reasoning):

> Knot theory provides rich invariants that capture complex topological properties. If these invariants are related to the complexity of Tseitin formulas, it could lead to new hardness results or efficient algorithms. The constructive mapping from a graph's knot invariant to its resolution proof length is expected to expose underlying structure that could be exploited for algorithmic purposes.

Taxonomy category: `TSEITIN_RES` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `50ad6a48a0591f4d`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if, for all n-vertex graphs G, the number of Seifert matrices (S(G)) is ≤ 2^n AND there exists a polynomial-time computable function f(n) such that resolution proof length (R(G)) for Tseitin formula on G is ≥ f(n). The conjecture is falsified if S(G) > 2^n OR R(G) < f(n) for any graph G.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	0.80	UNCERTAIN	HITS
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - 'Algebraic topology' AND 'knot invariants' AND 'resolution proof complexity' - 'Tseitin formulas' AND 'proof length' AND 'Seifert matrices' - 'Complexity theory' AND 'resolution proof complexity' AND 'knot theory'

Top relevant hits considered: - [http://arxiv.org/abs/math/9811006v1] Billiard knots in a cylinder - [http://arxiv.org/abs/math/0404143v1] Groups of locally-flat disk knots and non-locally-flat sphere knots - [http://arxiv.org/abs/0806.3452v2] Conjugate Generators of Knot and Link Groups - [http://arxiv.org/abs/2209.05839v3] On bounded depth proofs for Tseitin formulas on the grid; revisited - [http://arxiv.org/abs/2103.09609v1] Characterizing Tseitin-formulas with short regular resolution refutations - [http://arxiv.org/abs/1004.2159v2] Algebraic Proofs over Noncommutative Formulas - [http://arxiv.org/abs/2504.04416v1] Meta-Mathematics of Computational Complexity Theory - [http://arxiv.org/abs/1611.00827v2] Geometric complexity theory and matrix powering

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_graph(n):
        edges = []
        for i in range(n):
            for j in range(i + 1, n):
                if random.random() < 0.5:
                    edges.append((i, j))
        return edges

    def seifert_matrix(edges, n):
        M = [[0] * n for _ in range(n)]
        for u, v in edges:
            M[u][v] = -1
            M[v][u] = -1
            M[u][u] += 1
            M[v][v] += 1
        return M

    def resolution_length(edges):
        # Placeholder function; actual implementation needed
```

```
        return len(edges) ** 2

    n_values = [5, 10, 15, 20, 30, 40]
    results = []

    for n in n_values:
        graph_edges = generate_graph(n)
        S_G = seifert_matrix(graph_edges, n)
        R_G = resolution_length(graph_edges)

        results.append({
            "n": n,
            "S(G)": len(S_G),
            "R(G)": R_G
        })

    metric_value = sum(R_G / (2 ** n) for n, _, R_G in results)
    instances_tested = len(results)
    conjecture_holds = all(2 ** n >= S_G and R_G >= 10 * n for n, S_G, R_G in results)
    counterexample = "" if conjecture_holds else "mapping_undefined"

    return {
        "metric_name": "Mean Resolution Length / 2^n",
        "metric_value": metric_value,
        "instances_tested": instances_tested,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    results = []

    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    mean_value = sum(result["metric_value"] for result in results) / len(results)
    std_dev = math.sqrt(sum((result["metric_value"] - mean_value) ** 2 for result in results) / le
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_dev} support_fraction={support_fracti
    elif any(not result["conjecture_holds"] for result in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conj
        print(f"RESULT: FALSIFIED counterexample='mapping_undefined' first_failing_seed={first_fai
    else:
        print("RESULT: INCONCLUSIVE mapping_undefined")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_84d350e3.py", line 76, in <module>
    trial_result = run_trial(seed)
                   ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_84d350e3.py", line 56, in run_trial
    metric_value = sum(R_G / (2 ** n) for n, _, R_G in results)
                   ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_84d350e3.py", line 56, in <genexpr>
    metric_value = sum(R_G / (2 ** n) for n, _, R_G in results)
                               ~~^^~~
TypeError: unsupported operand type(s) for ** or pow(): 'int' and 'str'
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test crashed before producing data, which means we cannot verify whether the conjecture's conditions are met or not. | next: Re-run the test with proper error handling to ensure it completes without crashing and produces results.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	12396
2	propose	ollama_remote	glm4:latest	0	0	10702
3	preregistration	ollama_remote	glm4:latest	0	0	6094
4	novelty	ollama_remote	glm4:latest	0	0	4778
5	novelty	ollama_remote	glm4:latest	0	0	6132
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16806
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9227
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11423
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8596
10	judge	ollama_remote	glm4:latest	0	0	8004

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 94159 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/175abe514ed4.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/175abe514ed4.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/175abe514ed4.tar.gz` (if generated)